

목차 (Table of Contents)

1. RTSS 2022 Demand Layering 논문 심층 분석

- 1.1 기본 정보
- 1.2 핵심 아이디어
- 1.3 방법론 상세
- 1.4 실험 결과
- 1.5 한계점
- 1.6 Alpamayo 적용 가능성

2. GPU 메모리 최적화 관련 연구

- 2.1 모델 오프로딩 / CPU-GPU 파이프라이닝
- 2.2 레이어 단위 메모리 관리
- 2.3 양자화 기반 접근 (배제 - 참고용: 모델 정확도 저하 방식)
- 2.4 KV Cache 최적화
- 2.5 기타 접근법

3. 우리 연구와의 연결점

- 3.1 Alpamayo VRAM 최적화 전략 로드맵
- 3.2 Demand Layering의 Alpamayo 적용 시나리오
- 3.3 핵심 인사이트 요약

4. 참고 문헌 목록

- 핵심 논문 (Demand Layering 및 직접 관련)
- 모델 오프로딩 및 파이프라이닝
- 양자화 (배제 - 참고용)
- KV Cache 및 어텐션 최적화
- 메모리 관리 및 컴파일러
- VLM 서빙 및 멀티스테이지 파이프라인
- 프레임워크 및 실용 가이드

관련 연구 정리 - GPU 메모리 최적화 및 On-Demand Layering

조사일: 2026-02-25

1. RTSS 2022 Demand Layering 논문 심층 분석

1.1 기본 정보

항목	내용
논문 제목	Demand Layering for Real-Time DNN Inference with Minimized Memory Usage
저자	Mingoo Ji, Saehanseul Yi, Changjin Koo, Sol Ahn, Dongjoo Seo, Nikil Dutt, Jong-Chan Kim
발표처	43rd IEEE Real-Time Systems Symposium (RTSS 2022), December 2022
DOI	10.1109/RTSS55097.2022.00035
링크	arXiv , IEEE Xplore

1.2 핵심 아이디어

문제 정의: DNN 추론 시 모델 파라미터를 GPU 메모리에 **전부 로딩**한 후 실행하는 기존 방식은 GPU 메모리에 큰 부담을 준다. 특히 **통합 메모리 아키텍처(Unified Memory Architecture)**를 사용하는 임베디드 시스템에서는 CPU와 GPU가 메모리를 공유하므로, CPU 메모리를 스왑 장치로 활용하는 기존 기법이 적용 불가능하다.

핵심 해결책: 고속 SSD를 GPU의 **협력 파트너(co-running partner)**로 활용하여, DNN을 **레이어 단위로 로딩 및 실행**한다. 이를 통해 GPU 메모리 사용량을 **단일 레이어 수준**으로 최소화한다.

핵심 통찰: - DNN의 레이어별 순차 실행 특성을 이용하면, 모든 레이어를 동시에 메모리에 올릴 필요가 없다 - SSD에서 레이어 파라미터를 읽는 시간을 GPU 연산과 **파이프라인화**하면 지연을 최소화할 수 있다 - 전체 모델이 아닌 **현재 실행 중인 레이어 + 다음 레이어 프리페치** 분량만 메모리에 유지

1.3 방법론 상세

1.3.1 3단계 파이프라인 아키텍처

Demand Layering은 추론 지연을 세 가지 연산으로 분해한다:

1. **Read (R):** SSD에서 모델 파라미터를 CPU 메모리로 읽기 (DMA 전송)
2. **Copy (C):** CPU 메모리에서 GPU 메모리로 파라미터 복사 (PCIe/메모리 버스)
3. **Kernel (K):** GPU 커널로 DNN 레이어 실행 (GPU 연산)

이 세 단계를 **파이프라인으로 중첩(overlap)** 시켜, i 번째 레이어의 Kernel 실행과 동시에 $(i+1)$ 번째 레이어의 Read/Copy를 수행한다.

```

Layer i:   [ Read_i  ][ Copy_i  ][ Kernel_i ]
Layer i+1:           [ Read_i+1 ][ Copy_i+1 ][ Kernel_i+1 ]
Layer i+2:                   [ Read_i+2 ][ Copy_i+2 ][ Kernel_i+2 ]

```

1.3.2 정상 상태(Steady-State) 파이프라인

- 파이프라인이 안정화되면, **각 레이어의 실행 시간은 $\max(\text{Read}, \text{Copy}, \text{Kernel})$ 중 가장 긴 것에 의해 결정**
- GPU 연산이 Read/Copy보다 오래 걸리면, 로딩 오버헤드가 완전히 은닉(hidden)됨
- 반대의 경우에도 파이프라인 구조로 인해 오버헤드가 크게 감소

1.3.3 메모리 관리 전략

- GPU 메모리에는 **현재 레이어의 파라미터 + 활성화(activation) + 다음 레이어 프리페치 버퍼**만 유지
- 레이어 실행 완료 후 해당 파라미터 메모리를 **즉시 해제**하고 다음 레이어에 재사용
- **더블 버퍼링:** 두 개의 버퍼를 교대로 사용하여 로딩과 실행을 동시에 진행

1.4 실험 결과

메트릭	수치
평균 메모리 절감율	96.5%
평균 지연 오버헤드	14.8%
저지연 설정 메모리 절감율	88.4%
저지연 설정 오버헤드	< 1 ms

- 대표적 DNN 모델들에서 평가: 일반적인 CNN 및 딥러닝 모델 포함
- 임베디드 GPU 환경 (통합 메모리 아키텍처)에서 주로 평가
- NVMe SSD와 통합 GPU 간 직접 데이터 전송 활용
- 메모리 사용량을 단일 레이어 수준으로 줄이면서도 실시간 제약(deadline) 충족 가능

1.5 한계점

1. **SSD 의존성**: 고속 NVMe SSD가 필수적이며, SSD 속도가 파이프라인 성능의 상한을 결정
2. **임베디드 시스템 초점**: 통합 메모리 아키텍처(NVIDIA Jetson 등)에 최적화되어, 디스크리트 GPU(RTX 시리즈 등)에서의 적용 검증 부족
3. **단일 모델 추론**: 다중 DNN 동시 실행 시나리오 미고려
4. **LLM/VLM 미검증**: 논문 발표 시점(2022)에는 대규모 언어 모델 추론에 대한 검증 없음
5. **활성화 메모리 미포함**: 파라미터 메모리만 최적화하며, 중간 활성화(activation) 메모리는 별도 관리 필요
6. **SSD 수명 문제**: 반복적인 읽기 작업으로 SSD 내구성(wear-out) 영향 가능
7. **배치 처리**: 단일 배치(batch size 1) 위주의 실시간 추론에 초점

1.6 Alpamayo 적용 가능성

1.6.1 Alpamayo 구조 분석

Alpamayo는 세 단계의 순차적 파이프라인으로 구성된다: 1. **Vision Encoder**: 입력 이미지를 시각적 특성으로 인코딩 2. **VLM (Vision-Language Model, 8.2B 파라미터)**: 시각-언어 이해 및 추론 (가장 큰 컴포넌트) 3. **Diffusion Decoder (2.3B 파라미터)**: 출력 이미지 생성

12GB VRAM 환경에서, 이 세 모델을 동시에 메모리에 올리는 것은 불가능하다.

1.6.2 직접 적용 가능성

적용 대상	적합도	근거
Vision Encoder 레이어별 로딩	높음	상대적으로 작은 모델, 순차적 레이어 구조, CNN/ViT 기반이므로 원 논문의 시나리오와 유사
VLM 레이어별 로딩	중간	8.2B 파라미터의 Transformer 구조. 레이어별 로딩 가능하나 KV Cache 및 어텐션 메커니즘으로 인해 단순 적용 어려움
Diffusion Decoder 레이어별 로딩	중간-낮음	Diffusion 모델은 반복적(iterative) 디노이징 과정에서 동일 레이어를 여러 번 호출하므로, 매 스텝마다 레이어를 재로딩하는 비용이 크게 증가

1.6.3 수정/확장 적용 방안

- 모델 단위 순차 로딩 + 레이어 단위 최적화:** - 3개 모델을 순차적으로 실행하되, 각 모델 내에서 Demand Layering 적용 - Vision Encoder 실행 -> 결과 CPU에 저장 -> VLM 레이어별 로딩/실행 -> Diffusion Decoder 로딩/실행 - **예상 효과:** 12GB 내에서 가장 큰 모델(VLM 8.2B)도 실행 가능
- Diffusion 모델 적응형 캐싱:** - 모든 레이어를 매 디노이징 스텝마다 로딩하는 대신, 자주 호출되는 레이어는 GPU에 캐싱 - 덜 중요한 레이어만 on-demand 로딩하는 **하이브리드 전략**
- PCIe 대역폭 활용:** - 디스크리트 GPU 환경에서는 SSD 대신 **CPU RAM을 스왑 대상으로 활용** - CPU RAM <-> GPU 전송(PCIe 4.0 ~32GB/s)이 SSD 읽기(~7GB/s)보다 훨씬 빠름 - 원 논문의 파이프라인 아이디어를 CPU-GPU 오프로딩에 적용
- 양자화와의 결합:** - 4-bit 양자화 적용 시 VLM 8.2B -> ~4.1GB, Diffusion 2.3B -> ~1.2GB - 양자화된 레이어를 Demand Layering으로 로딩하면 전송량 4배 감소, 파이프라인 효율 극대화

1.6.4 예상 수치 분석

시나리오	VRAM 사용량 (추정)	지연 오버헤드 (추정)
전체 모델 FP16 로딩	~21 GB (불가)	기준
모델 단위 순차 로딩 (FP16)	~16.4 GB (VLM만으로 초과)	모델 전환 시 수 초
Demand Layering + FP16	~2-3 GB (단일 레이어 수준)	+15-30%
Demand Layering + INT4 양자화	~0.5-1 GB (단일 레이어)	+5-15%
모델 순차 + INT4 양자화 (레이어별 로딩 없음)	~4-5 GB	모델 전환 시 0.5-1 초

2. GPU 메모리 최적화 관련 연구

2.1 모델 오프로딩 / CPU-GPU 파이프라이닝

2.1.1 FlexGen (ICML 2023)

항목	내용
제목	FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU
핵심	GPU, CPU, 디스크의 3단계 메모리 계층을 활용한 오프로딩 프레임워크
방법	선형 프로그래밍(LP) 기반 최적 오프로딩 전략 탐색, 가중치/활성화/KV Cache의 최적 배치 결정
성과	OPT-175B를 16GB GPU에서 실행, 기존 대비 100배 높은 최대 처리량 달성
링크	arXiv , GitHub

Alpamayo 관련성: 높음. FlexGen의 LP 기반 오프로딩 전략을 Alpamayo의 3단계 파이프라인 (Vision Encoder -> VLM -> Diffusion)에 적용하여, 각 단계별 최적 메모리 배치 전략을 자동으로 탐색할 수 있다. 다만 FlexGen은 **처리량(throughput) 중심**이므로, Alpamayo의 **단일 요청 지연(latency)** 최적화에는 수정이 필요하다.

2.1.2 NEO (MLSys 2025)

항목	내용
제목	NEO: Saving GPU Memory Crisis with CPU Offloading for Online LLM Inference
핵심	어텐션 연산과 KV Cache를 GPU에서 CPU로 부분적으로 오프로딩
방법	비대칭 GPU-CPU 파이프라이닝, 부하 인식 스케줄링으로 GPU/CPU 자원 균형 활용
성과	T4 GPU에서 7.5배 처리량 향상, A10G에서 26%, H100에서 14% 향상
링크	arXiv , GitHub

Alpamayo 관련성: 높음. VLM 8.2B 모델의 어텐션 연산 중 일부를 CPU로 오프로딩하는 전략에 직접 활용 가능하다. 특히 KV Cache 오프로딩은 VLM의 장문 컨텍스트 처리 시 VRAM 절약에 효과적이다. 12GB GPU(RTX 3060 수준)에서 상당한 효과가 예상된다.

2.1.3 DeepSpeed ZeRO-Inference

항목	내용
제목	ZeRO-Inference: Democratizing Massive Model Inference
핵심	GPU 메모리보다 큰 모델을 CPU/NVMe로 완전히 오프로딩하여 단일 GPU에서 추론
방법	모든 가중치를 오프로딩하고, 통신-연산 중첩으로 성능 최적화. 4-bit 양자화 + KV Cache 오프로딩으로 추가 최적화
성과	추론 처리량 최대 20배 향상 (DeepSpeed >= 0.10.3)
링크	DeepSpeed Blog

Alpamayo 관련성: 높음. VLM 8.2B 파라미터 전체를 CPU로 오프로딩하고 레이어별로 GPU에서 실행하는 검증된 프레임워크. PyTorch 기반이므로 Alpamayo에 비교적 쉽게 통합 가능하다. 다만 완전 오프로딩은 지연이 크므로, 부분 오프로딩과 양자화를 조합하는 것이 바람직하다.

2.1.4 SwapAdvisor (ASPLOS 2020)

항목	내용
제목	SwapAdvisor: Pushing Deep Learning Beyond the GPU Memory Limit via Smart Swapping
저자	Chien-Chin Huang, Gu Jin, Jinyang Li (NYU)
핵심	데이터플로우 그래프 분석 기반, 연산자 스케줄링/메모리 할당/스왑 결정의 3차원 공동 최적화
방법	커스텀 유전 알고리즘으로 광대한 탐색 공간에서 최적 스왑 전략 탐색
성과	GPU 메모리 한계의 12배 큰 모델 학습, 이론적 최대 처리량의 53-99% 달성
링크	ACM

Alpamayo 관련성: 중간. 스왑 결정을 데이터플로우 그래프 수준에서 자동 최적화하는 아이디어는 Alpamayo의 복잡한 다단계 파이프라인에 유용하다. 다만 학습(training) 중심이므로 추론 파이프라인에 맞게 수정이 필요하다.

2.1.5 PIPO (arXiv 2025)

항목	내용
제목	PIPO: Pipelined Offloading for Efficient Inference on Consumer Devices
핵심	세밀한 오프로딩 파이프라인으로 소비자 디바이스에서 효율적 추론
방법	디스크-CPU-GPU 간 최적화된 데이터 전송 + 커스텀 양자화 커널 + 자동 구성
성과	RTX 3060 (6GB)에서 GPU 활용률 40% -> 90% 이상, 최대 3.1배 처리량 향상
링크	arXiv

Alpamayo 관련성: 매우 높음. RTX 3060 (6GB)라는 자원 제약 환경에서 검증된 기법이며, 12GB GPU 환경의 Alpamayo에 직접 적용 가능하다. 특히 디스크->CPU->GPU의 3단계 파이프라인 설계와 자동 최적 구성 기능은 Alpamayo의 다중 모델 환경에 매우 적합하다.

2.1.6 SpecOffload (arXiv 2025)

항목	내용
제목	SpecOffload: Unlocking Latent GPU Capacity for LLM Inference on Resource-Constrained Devices
핵심	오프로딩 파이프라인에 추측적 디코딩(speculative decoding)을 삽입하여 GPU 유휴 시간 활용
방법	이중 배치 회전(dual-batch rotation) 전략으로 검증/초안 생성을 동시 실행
성과	GPU 코어 활용률 4.49배 향상, 추론 처리량 2.54배 향상
링크	arXiv

Alpamayo 관련성: 중간. 오프로딩 중 GPU 유휴 시간을 활용하는 아이디어는 VLM 추론 시 레이어 로딩 대기 시간에 다른 연산을 삽입하는 데 응용 가능하다. 다만 VLM의 autoregressive 생성 부분에만 적용 가능하다.

2.1.7 Fiddler (ICLR 2025)

항목	내용
제목	Fiddler: CPU-GPU Orchestration for Fast Inference of Mixture-of-Experts Models
핵심	CPU의 연산 능력을 적극 활용하여 GPU-CPU 간 데이터 이동 최소화
방법	GPU에 없는 전문가 가중치를 CPU로 복사하는 대신, 활성화를 CPU로 복사하여 CPU에서 연산 후 결과를 GPU로 반환
성과	24GB GPU에서 비양자화 Mixtral-8x7B(90GB+) 실행, 3+ tokens/s 달성
링크	arXiv , GitHub

Alpamayo 관련성: 중간. “큰 가중치 대신 작은 활성화를 이동한다”는 발상의 전환이 핵심. Alpamayo에서 VLM의 일부 레이어를 CPU에서 실행하고 활성화만 GPU로 전송하는 하이브리드 전략에 참고할 수 있다.

2.1.8 Superpipeline (arXiv 2024)

항목	내용
제목	Superpipeline: A Universal Approach for Reducing GPU Memory Usage in Large Models
핵심	모델을 파티션 단위로 나누어 k개를 GPU에, 나머지를 CPU에 배치하는 동적 관리
방법	k (동시 GPU 로딩 파티션 수)와 k' (연산 후 CPU로 반환 파티션 수) 하이퍼파라미터로 메모리-속도 균형 조절
성과	GPU 메모리 사용량 최대 60% 감소, 모델 정확도 유지
적용 범위	LLM, VLM, Vision 모델 모두 지원
링크	arXiv

Alpamayo 관련성: 매우 높음. LLM, VLM, Vision 모델 모두에 적용 가능한 범용 프레임워크이며, 재학습이나 파라미터 변경 없이 기존 모델에 적용할 수 있다. Alpamayo의 세 모델 각각에 k , k' 파라미터를 다르게 설정하여 최적화할 수 있다.

2.2 레이어 단위 메모리 관리

2.2.1 LaLaRAND (RTSS 2021)

항목	내용
제목	LaLaRAND: Flexible Layer-by-Layer CPU/GPU Scheduling for Real-Time DNN Tasks
핵심	개별 DNN 레이어를 CPU 또는 GPU에 유연하게 할당하는 실시간 스케줄링
방법	CPU-친화적 양자화 + 동적 커널 디스패처 + 투명 메모리 핸들러. 레이어별 CPU/GPU 할당 최적화 알고리즘
성과	기존 대비 56-80% 더 많은 DNN 태스크셋 스케줄 가능, 정확도 손실 -0.4% 이내
링크	IEEE Xplore , GitHub

Alpamayo 관련성: 높음. 레이어별로 CPU/GPU를 동적으로 선택하는 접근법은 Alpamayo의 각 모델에서 연산 집약 레이어는 GPU에, 메모리 집약 레이어는 CPU에 할당하는 하이브리드 전략에 적용 가능하다. 특히 레이어별 양자화 수준을 다르게 적용하는 아이디어가 유용하다.

2.2.2 RT-Swap (RTAS 2024)

항목	내용
제목	RT-Swap: Addressing GPU Memory Bottlenecks for Real-Time Multi-DNN Inference
저자	Woosung Kang, Jinkyu Lee, Youngmoon Lee, Sangeun Oh, Kilho Lee, Hoon Sung Chwa
핵심	다중 DNN 동시 실행 시 GPU 메모리 부족 문제를 CPU 메모리 스왑으로 해결하며 실시간 보장
방법	연속 가상 주소(VA) 범위 유지 + 균일 물리 청크 할당으로 투명한 스왑 스케줄링
성과	기존 대비 최소 72% 더 많은 DNN 태스크셋 스케줄 가능 (메모리 수요가 물리 GPU 용량의 96.2% 초과 시에도)
링크	IEEE Xplore , GitHub

Alpamayo 관련성: 높음. Alpamayo의 세 모델이 순차적이 아닌 **부분적으로 동시에** 실행되어야 하는 경우(예: 파이프라인 중첩), RT-Swap의 실시간 보장 스왑 메커니즘이 VRAM 관리에 핵심적이다. 특히 CUDA VMM API를 활용한 구현은 PyTorch와 호환 가능하다.

2.2.3 Occamy (DAC 2023)

항목	내용
제목	Occamy: Memory-efficient GPU Compiler for DNN Inference
저자	Lee, J. et al. (Yonsei University)
핵심	DNN 컴파일러 수준에서 텐서의 생존 기간(liveness)을 분석하여 메모리 풀 최적 할당
방법	각 연산의 입출력 텐서 차원 및 생존 기간 분석 -> 최대 필요 메모리 크기 계산 -> 최적 텐서 배치 스케줄링
성과	PyTorch 대비 메모리 사용 34.6% 감소, 1.25배 속도 향상 (임베디드 GPU)
링크	IEEE Xplore , GitHub

Alpamayo 관련성: 중간. 컴파일러 수준의 텐서 메모리 최적화는 Alpamayo 각 모델의 중간 활성화 메모리를 줄이는 데 도움이 된다. 특히 Diffusion Decoder의 반복적 디노이징 과정에서 활성화 메모리 재사용을 극대화할 수 있다.

2.2.4 GPU Memory Oversubscription via NVMe Paging (RTSS 2022)

항목	내용
제목	Enabling GPU Memory Oversubscription via Transparent Paging to an NVMe SSD
발표처	RTSS 2022 (Demand Layering과 같은 학회)
핵심	GPU 메모리 버퍼를 CPU 메모리가 아닌 NVMe SSD로 직접 페이징
성과	기존 demand paging 대비 3배 빠른 종단 간 성능, 81% 낮은 오버헤드
링크	PDF , IEEE Xplore

Alpamayo 관련성: 중간. 디스크리트 GPU 환경에서 GPU 메모리 초과 사용 시 NVMe SSD로의 투명한 페이징 메커니즘을 제공한다. Demand Layering과 조합하면 더 큰 모델을 더 작은 GPU에서 실행할 수 있는 기반 기술이 된다.

2.3 양자화 기반 접근 (배제 - 참고용: 모델 정확도 저하 방식)

2.3.1 AWQ (MLSys 2024 Best Paper) (배제 - 참고용)

항목	내용
제목	AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration
핵심	활성화 분포를 관찰하여 중요한(salient) 가중치를 보호하는 가중치 양자화
방법	역전파나 재구성 없이 활성화 영향력 기반으로 중요 가중치 식별 및 보호
성과	4-bit 양자화로 FP16 대비 메모리 4배 절감, 일반화 능력 우수
적용	TensorRT-LLM, vLLM, HuggingFace TGI, LMDeploy 등에 통합
링크	GitHub

Alpamayo 관련성: 매우 높음. VLM 8.2B를 4-bit AWQ 양자화하면 ~4.1GB -> ~2GB로 줄어든다. Diffusion Decoder 2.3B도 ~1.2GB -> ~0.6GB. 이를 Demand Layering/오프로딩과 조합하면 12GB VRAM에서 충분히 실행 가능해진다. AWQ는 캘리브레이션 데이터 요구량이 적어 적용이 용이하다.

2.3.2 GPTQ (배제 - 참고용)

항목	내용
제목	GPTQ: Accurate Post-Training Quantization for Generative Pre-Trained Transformers
핵심	Hessian 기반 레이어별 후학습 양자화로 출력 오차 최소화
방법	레이어 단위로 2차 정보(Hessian)를 활용하여 최적 양자화 파라미터 탐색
성과	4-bit에서 FP16과 거의 동등한 품질, GPU 추론에 최적화 (Marlin 커널로 GGUF 대비 5배 빠름)

Alpamayo 관련성: 높음. GPU 추론에 최적화된 양자화 기법으로, VLM과 Diffusion Decoder 모두에 적용 가능하다. 특히 Marlin 커널과의 조합은 양자화된 모델의 GPU 추론 속도를 극대화한다.

2.3.3 SqueezeLLM (ICML 2024) (배제 - 참고용)

항목	내용
제목	SqueezeLLM: Dense-and-Sparse Quantization
핵심	2차 정보 기반 비균일 양자화 + 이상치(outlier)를 희소 형식으로 별도 저장
방법	민감도 기반 비트 정밀도 할당 + Dense-and-Sparse 분해로 극저정밀(3-bit)까지 무손실 압축
성과	A6000에서 최대 2.3배 속도 향상, Vicuna 모델 6GB 내 서빙 가능
링크	arXiv , GitHub

Alpamayo 관련성: 높음. 3-bit 양자화로 VLM 8.2B를 ~3GB 수준으로 압축 가능. 특히 이상치 가중치를 희소 형식으로 별도 관리하는 기법은 VLM의 품질 유지에 효과적이다.

2.3.4 Q-VLM (NeurIPS 2024) (배제 - 참고용)

항목	내용
제목	Q-VLM: Post-training Quantization for Large Vision-Language Models
핵심	VLM 특화 후학습 양자화, 교차 레이어 의존성을 고려한 양자화 전략
방법	활성화 엔트로피를 프록시로 교차 레이어 의존성 분석 -> 최적 블록 파티셔닝 -> W4A4 양자화
성과	13B LLaVA 모델에서 메모리 2.78배 압축, 생성 속도 1.44배 향상, 성능 저하 없음
링크	arXiv , GitHub

Alpamayo 관련성: 매우 높음. Alpamayo의 VLM 컴포넌트에 **직접 적용 가능한** VLM 특화 양자화 기법이다. Vision Encoder와 Language Model 간의 교차 레이어 의존성을 고려하므로, 일반 LLM 양자화보다 VLM에서의 품질 유지가 우수하다. 8.2B VLM을 W4A4로 양자화하면 ~2GB 수준으로 압축 가능하다.

2.4 KV Cache 최적화

2.4.1 vLLM / PagedAttention (SOSP 2023)

항목	내용
제목	Efficient Memory Management for Large Language Model Serving with PagedAttention
핵심	OS의 가상 메모리 페이징 개념을 KV Cache 관리에 적용
방법	KV Cache를 고정 크기 블록으로 분할, 논리-물리 블록 매핑 테이블로 관리, 메모리 공유 지원
성과	KV Cache 메모리 낭비 60-80% -> 4% 미만, 처리량 2-4배 향상
링크	arXiv , GitHub

Alpamayo 관련성: 높음 (VLM 부분에 한정). Alpamayo VLM의 autoregressive 생성 단계에서 KV Cache가 VRAM을 상당량 차지한다. PagedAttention으로 KV Cache 메모리 효율을 극대화하면, 나머지 VRAM을 모델 가중치에 더 많이 할당할 수 있다. 다만 Alpamayo가 batch size 1 위주라면 효과가 제한적이다.

2.4.2 KV Cache 오프로딩 기법들

기법	핵심	Alpamayo 관련성
NVIDIA KV Cache Offload	NVLink-C2C로 KV Cache를 CPU 메모리로 오프로딩, Grace Hopper/Blackwell 아키텍처 최적화	중간 (특정 하드웨어 의존)
DeepSpeed KV Cache Offload	KV Cache를 CPU 메모리로 이동, GPU 메모리 요구량 대폭 감소	높음 (범용 적용 가능)
Streaming KV Cache	가장 최근/중요한 토큰의 KV만 GPU에 유지	높음 (VRAM 절약에 직접적)

2.5 기타 접근법

2.5.1 Diffusers 메모리 최적화 기법 (HuggingFace)

HuggingFace Diffusers 라이브러리는 다양한 메모리 최적화 기법을 기본 제공한다:

기법	설명	메모리 절감	속도 영향
<code>enable_model_cpu_offload()</code>	전체 모델 단위로 CPU-GPU 간 오프로딩	중간	약간 느림
<code>enable_sequential_cpu_offload()</code>	서브모듈 단위 세밀한 오프로딩	높음	매우 느림
<code>enable_group_offload()</code>	레이어 그룹 단위 오프로딩 (block_level / leaf_level)	높음	중간
Group Offload + CUDA Stream	비동기 스트림으로 다음 레이어 프리페치	높음	빠름
<code>enable_vae_tiling()</code>	VAE를 타일 단위로 분할 처리	중간	약간 느림
<code>enable_vae_slicing()</code>	배치를 개별 이미지로 분할 디코딩	중간 (다중 이미지 시)	미미
<code>enable_layerwise_casting()</code>	FP8 저장 + FP16/BF16 연산으로 메모리 절약	중간 (~2배)	미미
Offload to Disk	CPU 메모리 부족 시 디스크로 추가 오프로딩	매우 높음	느림

Alpamayo 관련성: 매우 높음. Diffusion Decoder에 직접 적용 가능한 검증된 기법들이다. 특히: - **Group Offload + CUDA Stream:** Demand Layering과 유사한 파이프라인 효과를 Diffusers 프레임워크에서 네이티브로 제공 - **Layerwise Casting:** FP8 저장으로 모델 메모리를 절반으로 줄이면서 연산 품질 유지 - **model_cpu_offload + group_offload 조합:** Vision Encoder -> VLM -> Diffusion 순차 실행에 최적

2.5.2 Nova (2025)

항목	내용
제목	Nova: Real-Time Agentic Vision-Language Model Serving with Adaptive Cross-Stage Parallelization
핵심	단일 GPU에서 VLM 다단계 파이프라인(Vision Encode -> LLM Prefill -> LLM Decode)의 실시간 서빙
방법	적응형 교차 단계 파이프라인 병렬화 + SM 파티셔닝 + Vision Encoder 가중치 비동기 오프로딩
성과	평균 지연 14.6%, 최대 지연 23.3% 개선
링크	arXiv

Alpamayo 관련성: 매우 높음. Alpamayo와 거의 동일한 구조(Vision Encoder -> LLM -> 출력)를 가진 VLM의 서빙 최적화 프레임워크이다. 특히: - Vision Encoder 가중치를 CPU-GPU 간 비동기 스왑하는 기법이 Alpamayo에 직접 적용 가능 - SM 파티셔닝으로 GPU 자원을 단계별로 유연하게 분할하는 전략 참고 가능 - 다만 Nova는 Diffusion Decoder를 포함하지 않으므로, 이 부분은 별도 최적화 필요

2.5.3 SmolVLM (arXiv 2025)

항목	내용
제목	SmolVLM: Redefining Small and Efficient Multimodal Models
핵심	모델 자체를 극소형(256M-2.2B)으로 설계하여 자원 효율적 추론
방법	Vision Encoder/Language Model 최적 크기 배분, 공격적 토큰 압축(pixel shuffle), 데이터 큐레이션
성과	SmolVLM-256M: 1GB 미만 GPU 메모리로 Idefics-80B 능가
링크	arXiv

Alpamayo 관련성: 중간-높음. Alpamayo가 자체 모델 크기를 조절할 수 있다면, SmolVLM의 아키텍처 효율화 원칙(토큰 압축, 최적 인코더-디코더 비율)을 참고하여 VLM 컴포넌트를 경량화할 수 있다. 다만 기존 모델을 그대로 사용해야 하는 경우에는 직접 적용이 어렵다.

2.5.4 GMLake (ASPLOS 2024)

항목	내용
제목	GMLake: Efficient and Transparent GPU Memory Defragmentation for Large-scale DNN Training with Virtual Memory Stitching
핵심	비연속 메모리 블록을 가상 메모리 매핑으로 결합하여 단편화 해결
성과	평균 9.2GB (최대 25GB) GPU 메모리 절감, 15% (최대 33%) 단편화 감소 (A100 80GB)
링크	arXiv , GitHub

Alpamayo 관련성: 중간. 12GB VRAM에서 다중 모델을 순차적으로 로딩/해제할 때 발생하는 메모리 단편화를 해결하는 데 유용하다. 모델 전환 시 단편화로 인한 OOM(Out-Of-Memory) 에러를 방지할 수 있다.

2.5.5 HERMES (arXiv 2025)

항목	내용
제목	Understanding and Optimizing Multi-Stage AI Inference Pipelines
핵심	다단계 LLM 추론 파이프라인의 이산 이벤트 시뮬레이터
방법	RAG, KV 검색, 추론, prefill, decode 등 다양한 단계를 모델링하며 하드웨어 계층(GPU, CPU, 메모리) 간 최적 구성 탐색
링크	arXiv

Alpamayo 관련성: 중간. Alpamayo의 다단계 파이프라인 성능을 시뮬레이션하고 최적 구성을 탐색하는 도구로 활용 가능하다. 각 단계의 배치 전략, 메모리 계층 선택, 병렬화 구성을 사전에 시뮬레이션할 수 있다.

2.5.6 Flash Attention

항목	내용
제목	FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness
핵심	타일링 기법으로 어텐션을 작은 블록 단위로 연산, HBM 접근 최소화
성과	어텐션 메모리 $O(N^2) \rightarrow O(N)$, 2-4배 속도 향상

Alpamayo 관련성: 높음. VLM 8.2B의 어텐션 레이어에서 활성화 메모리를 크게 줄인다. 대부분의 현대 프레임워크(PyTorch 2.0+)에서 SDPA로 자동 적용되므로, 추가 구현 없이도 효과를 얻을 수 있다.

3. 우리 연구와의 연결점

3.1 Alpamayo VRAM 최적화 전략 로드맵

12GB VRAM에서 Alpamayo(Vision Encoder + VLM 8.2B + Diffusion Decoder 2.3B)를 실행하기 위한 최적화 전략을 우선순위별로 정리한다:

Tier 1: 즉시 적용 가능한 기법 (낮은 구현 비용)

기법	참고 연구	예상 효과	구현 난이도
모델 순차 CPU 오프로딩	Diffusers enable_model_cpu_offload	전체 VRAM을 최대 모델 1개 크기로 제한	매우 쉬움
4-bit 양자화 (AWQ/GPTQ)	AWQ, GPTQ, Q-VLM	VLM: ~16GB->~4GB, Diff: ~4.6GB->~1.2GB	쉬움
Flash Attention	FlashAttention	어텐션 활성화 메모리 대폭 감소	자동 적용
VAE Tiling	Diffusers enable_vae_tiling	Diffusion Decoder 활성화 메모리 감소	매우 쉬움
FP8 Layerwise Casting	Diffusers enable_layerwise_casting	모델 가중치 메모리 ~2배 감소	쉬움

Tier 1 적용 시 예상: VLM(4-bit) ~4GB + 활성화 ~2-3GB + KV Cache ~1-2GB = 약 7-9GB. 12GB 이내 가능성 높음.

Tier 2: 중간 수준 최적화 (중간 구현 비용)

기법	참고 연구	예상 효과	구현 난이도
Group Offloading + CUDA Stream	Diffusers group_offload, Demand Layering	CPU 프리페치로 오프로딩 지연 은닉	중간
KV Cache 오프로딩	NEO, DeepSpeed	VLM의 KV Cache를 CPU로 이동, VRAM 절약	중간
Superpipeline 적용	Superpipeline	각 모델 내부 파티션별 GPU/CPU 동적 관리	중간
VLM 특화 양자화	Q-VLM	VLM의 교차 레이어 의존성 고려한 고품질 양자화	중간

Tier 2 추가 적용 시 예상: VLM 실행 시 ~4-6GB VRAM으로 충분, Diffusion과 동시 실행도 가능성 있음.

Tier 3: 고급 최적화 (높은 구현 비용)

기법	참고 연구	예상 효과	구현 난이도
Demand Layering 직접 구현	RTSS 2022 Demand Layering	레이어 단위 로딩으로 VRAM 극소화 (~1-2GB)	높음
CPU-GPU 하이브리드 연산	Fiddler, LaLaRAND	일부 레이어를 CPU에서 실행	높음
LP 기반 자동 오프로딩 전략	FlexGen	다단계 파이프라인 최적 메모리 배치 자동 탐색	높음
Nova식 SM 파티셔닝	Nova	GPU SM을 단계별로 분할하여 동시 실행	매우 높음

3.2 Demand Layering의 Alpamayo 적용 시나리오

Demand Layering의 핵심 아이디어를 Alpamayo에 적용하는 구체적 시나리오:

[시나리오 A: 순수 Demand Layering 방식]

- Vision Encoder 레이어 1~N을 SSD/CPU에서 한 레이어씩 GPU에 로딩/실행
 - 메모리: ~50-100MB (단일 레이어)
 - 파이프라인으로 로딩 지연 은닉
- Vision Encoder 결과를 CPU RAM에 저장 (수 MB)
- VLM 8.2B 레이어 1~M을 한 레이어씩 GPU에 로딩/실행
 - 메모리: ~100-200MB (단일 레이어, INT4 기준)
 - KV Cache는 GPU에 점진적 구축 또는 CPU 오프로딩
 - 파이프라인으로 로딩 지연 은닉
- VLM 결과를 CPU RAM에 저장
- Diffusion Decoder 2.3B 실행
 - 각 디노이징 스텝마다 모든 레이어 순회
 - 50 스텝 x 레이어 수만큼 반복 로딩 필요
 - --> 여기가 Demand Layering의 약점: 반복 로딩 오버헤드

VRAM 사용: ~1-2GB (최소)

지연 오버헤드: Vision/VLM에서는 15-30%, Diffusion에서는 100%+ 가능

[시나리오 B: 하이브리드 방식 (권장)]

1. Vision Encoder: Group Offloading + CUDA Stream (Tier 2)
 - VRAM: ~0.5-1GB
2. VLM 8.2B: 4-bit 양자화 + KV Cache 오프로딩 (Tier 1+2)
 - VRAM: ~4-5GB (양자화된 전체 모델)
 - 또는 Demand Layering + 양자화: ~0.5-1GB
3. Diffusion Decoder 2.3B: 4-bit 양자화 + 전체 GPU 로딩 (Tier 1)
 - VRAM: ~1.2GB (반복 디노이징에 최적)
4. 각 단계 간 model_cpu_offload로 전환

총 피크 VRAM: $\max(\text{각 단계}) = \sim 5-6\text{GB}$ (12GB 내 충분)
지연 오버헤드: ~20-40% (모델 전환 포함)

3.3 핵심 인사이트 요약

1. Demand Layering은 “레이어별 순차 로딩”의 이론적 기반을 제공하지만, Diffusion 모델의 반복적 특성에는 직접 적용이 어렵다.
2. 가장 실용적인 조합: 4-bit 양자화(AWQ/Q-VLM) + 모델 단위 순차 오프로딩(Diffusers) + Group Offloading with CUDA Stream. 이 조합으로 12GB VRAM에서 ~5-6GB 피크 사용으로 실행 가능.
3. VLM이 병목: 8.2B 파라미터가 가장 큰 메모리 부담. 양자화가 필수적이며, KV Cache 관리가 부차적으로 중요하다.
4. Diffusion Decoder는 반복 실행이 특수: 다른 컴포넌트와 달리, 동일 가중치가 수십 번 반복 사용되므로 GPU에 유지하는 것이 합리적. 양자화로 크기를 줄여 상시 GPU에 탑재하는 전략이 최적.
5. CPU-GPU 대역폭이 핵심 제약: PCIe 4.0 x16 (~32GB/s)에서 FP16 기준 VLM 전체 로딩에 ~0.5초, INT4 기준 ~0.13초. 모델 전환 시 이 지연을 허용할 수 있다면 순차 실행이 가장 단순한 해법.

4. 참고 문헌 목록

핵심 논문 (Demand Layering 및 직접 관련)

1. M. Ji, S. Yi, C. Koo, S. Ahn, D. Seo, N. Dutt, and J.-C. Kim, “Demand Layering for Real-Time DNN Inference with Minimized Memory Usage,” in Proc. RTSS, 2022.
[arXiv](#)

2. W. Kang, J. Lee, Y. Lee, S. Oh, K. Lee, and H. S. Chwa, "RT-Swap: Addressing GPU Memory Bottlenecks for Real-Time Multi-DNN Inference," in Proc. RTAS, 2024. [IEEE](#)
3. K. Lee et al., "LaLaRAND: Flexible Layer-by-Layer CPU/GPU Scheduling for Real-Time DNN Tasks," in Proc. RTSS, 2021. [IEEE](#)
4. J. Bakita et al., "Enabling GPU Memory Oversubscription via Transparent Paging to an NVMe SSD," in Proc. RTSS, 2022. [PDF](#)

모델 오프로딩 및 파이프라이닝

1. Y. Sheng et al., "FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU," in Proc. ICML, 2023. [arXiv](#)
2. C. Xu et al., "NEO: Saving GPU Memory Crisis with CPU Offloading for Online LLM Inference," in Proc. MLSys, 2025. [arXiv](#)
3. DeepSpeed Team, "ZeRO-Inference: Democratizing Massive Model Inference," 2022. [Blog](#)
4. C.-C. Huang, G. Jin, and J. Li, "SwapAdvisor: Pushing Deep Learning Beyond the GPU Memory Limit via Smart Swapping," in Proc. ASPLOS, 2020. [ACM](#)
5. Y. Li et al., "PIPO: Pipelined Offloading for Efficient Inference on Consumer Devices," arXiv:2504.03664, 2025. [arXiv](#)
6. X. Zhang et al., "SpecOffload: Unlocking Latent GPU Capacity for LLM Inference on Resource-Constrained Devices," arXiv:2505.10259, 2025. [arXiv](#)
7. K. Kamahori et al., "Fiddler: CPU-GPU Orchestration for Fast Inference of Mixture-of-Experts Models," in Proc. ICLR, 2025. [arXiv](#)
8. K. Thorat et al., "Superpipeline: A Universal Approach for Reducing GPU Memory Usage in Large Models," arXiv:2410.08791, 2024. [arXiv](#)

양자화 (배제 - 참고용)

1. J. Lin et al., "AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration," in Proc. MLSys, 2024 (Best Paper). [GitHub](#)
2. E. Frantar et al., "GPTQ: Accurate Post-Training Quantization for Generative Pre-Trained Transformers," in Proc. ICLR, 2023.
3. S. Kim et al., "SqueezeLLM: Dense-and-Sparse Quantization," in Proc. ICML, 2024. [arXiv](#)

4. C. Wang et al., “Q-VLM: Post-training Quantization for Large Vision-Language Models,” in Proc. NeurIPS, 2024. [arXiv](#)

KV Cache 및 어텐션 최적화

1. W. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” in Proc. SOSP, 2023. [arXiv](#)
2. T. Dao et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” in Proc. NeurIPS, 2022.

메모리 관리 및 컴파일러

1. J. Lee et al., “Occamy: Memory-efficient GPU Compiler for DNN Inference,” in Proc. DAC, 2023. [IEEE](#)
2. Z. Guo et al., “GMLake: Efficient and Transparent GPU Memory Defragmentation for Large-scale DNN Training with Virtual Memory Stitching,” in Proc. ASPLOS, 2024. [arXiv](#)

VLM 서빙 및 멀티스테이지 파이프라인

1. J. Chen et al., “Nova: Real-Time Agentic Vision-Language Model Serving with Adaptive Cross-Stage Parallelization,” arXiv:2509.21301, 2025. [arXiv](#)
2. A. Maiti et al., “SmolVLM: Redefining Small and Efficient Multimodal Models,” arXiv:2504.05299, 2025. [arXiv](#)
3. S. Raghuraman et al., “Understanding and Optimizing Multi-Stage AI Inference Pipelines (HERMES),” arXiv:2504.09775, 2025. [arXiv](#)

프레임워크 및 실용 가이드

1. HuggingFace, “Reduce Memory Usage (Diffusers),” 2025. [Docs](#)
2. NVIDIA, “Cut Model Deployment Costs While Keeping Performance with GPU Memory Swap,” 2024. [Blog](#)

참고: 이 문서는 Alpamayo 모델의 12GB VRAM 환경 최적화 연구를 위해 작성되었다. 각 논문의 상세 내용은 원문을 참조하며, 적용 가능성 분석은 공개된 정보를 기반으로 한 추정치를 포함한다.